- **Oscar Poveda**
- **Elian Ibarra**

RETROSPECTIVA VERSIÓN UNO DOPOSoccer

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

R: La versión uno (Presentación + Dominio) se dividió en los siguientes mini-ciclos:

MINI-CICLO	OBJETIVO / ENTREGABLES	JUSTIFICACIÓN
1 **21 – 24 oct**	**Bosquejo del diseño: diagrama de clases en Astah y mockup de la interfaz en Figma.**	Era el entregable de Estructura y lo primero antes de programar. En Astah definimos las clases, la fachada DOPOSoccer y las jerarquías de modos, tipos de partido, poderes y efectos, pensadas para extender el juego. En Figma diseñamos todas las pantallas para tener la misma referencia visual.
2 **5 – 7 nov**	**Maqueta de la presentación: vistas de inicio y configuración completa del Arcade.**	La versión uno pide una maqueta con la configuración del juego. Separamos la GUI en clases de vista con métodos comunes y DOPOSoccerGUI quedó como controlador, así cada pantalla se construía y probaba por separado.
3 **7 nov**	**Estructura del dominio y código mínimo para jugar jugador vs jugador.**	**El nivel básico exige un partido PvP por tiempo y por goles. Creamos toda la estructura del diagrama, pero solo con el código necesario. Las opciones de versiones siguientes lanzan "en construcción", y la física se resolvió con dyn4j dentro de PhysicsEngine.**
4 **7 nov**	**Conexión de la presentación con el dominio.**	**La interfaz no debe conocer las reglas del juego. La GUI pide a la fachada todo lo que pinta (jugadores, balón, marcador, tiempo), envía las teclas de cada jugador y muestra los errores de DOPOSoccerException.**
5 **7 nov**	**Reglas del partido: puntos, muerte súbita, poderes, efectos, aturdimiento y física.**	**Con el partido funcionando completamos las reglas del enunciado y del Anexo 1, y ajustamos la física porque el balón no se comportaba como en el juego original.**
6 **7 – 8 nov**	**Sonido, música, volumen, vistas de torneo y tienda, y log de errores.**	**El dominio registra los eventos del partido y la GUI los reproduce como sonido; los volúmenes se guardan en Settings. La clase Log cumple el requisito del log de errores, y las vistas de torneo y tienda adelantan la versión dos.**

MINI-CICLO	OBJETIVO / ENTREGABLES	JUSTIFICACIÓN
7 8 nov	Diagrama de clases actualizado, diagramas de secuencia y pruebas de unidad.	Se dejó para el final para que los diagramas reflejaran el código terminado. Sincronizamos el diagrama de clases, hicimos los diagramas de secuencia de los métodos principales de la fachada y escribimos 102 pruebas con JUnit 5, que pasan todas.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?
   R: Los mini-ciclos 1 a 6 están terminados: la maqueta tiene toda la configuración del juego y el nivel básico jugador vs jugador funciona por tiempo y por goles con todas las reglas de poderes, efectos, aturdimiento y puntos. Las opciones de las versiones siguientes ya aparecen en la interfaz y responden "en construcción". Del mini-ciclo 7, el diagrama de clases y las 102 pruebas de unidad están completos, y los diagramas de secuencia están también: ya están los de los métodos principales de la fachada y faltan algunos métodos secundarios. Esto se logró porque cada mini-ciclo dejaba algo que se podía ejecutar y probar antes de pasar al siguiente.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes?
   Elián Ibarra: 44 horas.
   Oscar Poveda: 44 horas.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?
   R: El mayor logro fue tener un partido jugable jugador vs jugador que respeta las reglas del enunciado, con la presentación totalmente separada del dominio. La interfaz no tiene lógica del juego: todo lo pide a la fachada DOPOSoccer, y por eso pudimos trabajar en paralelo, uno en la interfaz y otro en el dominio, y unir las dos partes sin reescribirlas. Además, dejar desde ahora todas las opciones de las versiones siguientes con sus firmas definitivas hace que las próximas versiones solo tengan que completar código y no cambiar la interfaz.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
   R: El mayor problema técnico fue la física del partido. Al principio el balón y los jugadores no se comportaban como en el juego original: los valores de gravedad, rebote y velocidad no daban una sensación

natural, los choques con el travesaño no se detectaban bien y un tiro muy rápido podía atravesar a un jugador. Lo resolvimos usando la librería dyn4j, encerrada en la clase PhysicsEngine para que el resto del dominio no dependa de ella, y ajustando los valores con pruebas en el juego hasta que el comportamiento fue el esperado. Otro problema fue el trabajo en paralelo sobre el archivo de Astah: al ser binario, Git no puede mezclar los cambios de los dos, así que acordamos que solo uno lo edita a la vez y el otro le pasa los cambios.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
   R: Lo mejor que hicimos fue dividir el trabajo por capas desde el diseño: definimos juntos el diagrama de clases y la fachada, y después cada uno avanzó en su parte sabiendo exactamente qué métodos iba a usar o a implementar. También trabajamos con ramas separadas y pull requests, lo que nos permitió revisar los cambios del otro antes de unirlos. Consideramos que hicimos lo mejor posible, pues los requerimientos se enviaron muy tarde y la fecha de entrega estaba muy encima, lo que nos obligó a organizar muy bien el poco tiempo que tuvimos y extendernos mucho incluyendo el desorden de horario con tal de poder tener todo en la fecha.

7. Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?
   R: La práctica XP más útil fueron las entregas pequeñas (Small Releases). Cada mini-ciclo terminaba con algo que se podía ejecutar: primero la maqueta navegable, luego un partido mínimo, después la conexión con el dominio y por último las reglas completas. Así detectábamos los errores pronto y siempre teníamos una versión funcional que mostrar. Consideramos que dimos lo mejor de nosotros: los requerimientos se publicaron muy tarde y la fecha de entrega estaba muy cerca, así que tuvimos que organizar muy bien el poco tiempo disponible y dedicarle muchas horas adicionales, incluso sacrificando nuestros horarios habituales más de lo que deberíamos, para tener todo listo a tiempo.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

- Oracle. (2024). Creating a GUI With Swing (The Java Tutorials). https://docs.oracle.com/javase/tutorial/uiswing/
- Oracle. (2024). A Visual Guide to Layout Managers. https://docs.oracle.com/javase/tutorial/uiswing/layout/visual.html
- dyn4j. (2024). dyn4j: Java Collision Detection and Physics Engine (versión 5.0.2). https://dyn4j.org/
- The JUnit Team. (2024). JUnit 5 User Guide. https://junit.org/junit5/docs/current/user-guide/
- Change Vision. (2024). Astah UML: Manual de usuario. https://astah.net/manual/
- Figma. (2024). Figma Help Center. https://help.figma.com/
- Head Soccer Wiki. (s. f.). Fight Mode. https://head-soccer.fandom.com/
- Anthropic. (2026). Claude Opus 5.5 [Modelo de lenguaje de gran tamaño]. https://www.anthropic.com

Referencia más útil: la más útil fue el tutorial de Swing de Oracle, en especial la guía de los administradores de diseño (BorderLayout, FlowLayout, GridLayout), porque toda la maqueta se construyó con paneles anidados y entender cómo cada layout reparte el espacio fue lo que más tiempo nos ahorró. La IA generativa también fue útil para revisar errores de compilación, entender la API de dyn4j y apoyar la elaboración de los diagramas de secuencia.